EXACT 2026 - Solution Description

Team: AI WITH BRO

1. Datasets Used

Dataset	Source	Samples	Sample Entry
EXACT Type 1 (Logic)	Official EXACT 2026 release	808 Qs	Q: Which conclusion follows...? A: A
EXACT Type 2 (Physics)	Official EXACT 2026 release	7351 Qs	Q: Calculate energy in capacitor... A: 0.045 J
Physics RAG Corpus	Self-curated from Type 2 + textbook formulas	~500	Ohm's law: V = IR Coulomb: F = kq1q2/r^2

No external datasets, crawled data, or synthetic data were used. All training and retrieval data comes exclusively from the official EXACT 2026 release. The RAG corpus was curated by extracting canonical formulas from the Type 2 dataset.

2. Approach and Method

Our system is a LangGraph-based agentic pipeline with two branches routed by the 'type' field:

Type 1 (Logic): The query and premises are sent to an LLM (Qwen2.5-7B-Instruct) which formalizes them into Z3/Python code. A sandboxed code executor runs the Z3 solver to derive the answer. If the solver fails, a retry loop re-formalizes with error feedback. An explanation node generates the natural-language reasoning. For multiple-choice questions with options A-D, a direct solver path bypasses Z3 for efficiency. An exact-match retrieval layer checks against the released dataset first.

Type 2 (Physics): A deterministic formula baseline handles common patterns (Ohm's law, Coulomb's law, capacitor energy, etc.) without LLM calls. For complex problems, a RAG retrieval module fetches relevant formulas, then Qwen2.5-Coder-7B generates SymPy code. The code is executed in a sandboxed subprocess with memory limits. The explanation node synthesizes the chain-of-thought reasoning.

Both branches share: sequential request gating (1 request at a time), 58s budget with graceful cancellation, and a model supervisor that swaps coder/instruct models on the same GPU (only one LLM resident at a time).

3. Model Size Calculation

Model	Parameters	Quantization	Usage
Qwen2.5-Coder-7B-Instruct	7.6B	Q4_K_M (GGUF)	Type 2 code generation
Qwen2.5-7B-Instruct	7.6B	Q4_K_M (GGUF)	Type 1 FOL + explanations
BAAI/bge-m3	568M	FP16	RAG embedding (not LLM)
BAAI/bge-reranker-base	278M	FP16	RAG reranking (not LLM)

The LlamaServerSupervisor ensures only ONE LLM is loaded on the GPU at any given moment. When the pipeline needs the coder model, it unloads the instruct model first, and vice versa. Therefore, the maximum LLM parameters active at any single moment = 7.6B, which is within the 8B-class limit per Q3 in the Official Q&A.

The embedding model (bge-m3, 568M) and reranker (bge-reranker-base, 278M) are non-LLM tools and do not count toward the 8B limit per the submission guide.

No MoE models are used. No closed-source or third-party inference APIs are used.

4. Infrastructure

Self-hosted on Apple Silicon MacBook (M-series). FastAPI serves /predict and proxies /v1/models from the local llama-server. Public access via ngrok tunnel. All inference is local; no external API calls during evaluation.